

18 案例三：独立开发者一周做出AI 小产品——从 MVP 到能用的完整过程

背景：一个真实的"读书类" AI 小产品

转型做独立 AI 产品手艺人之后，我做过一个叫"读书行动派"的小产品：输入书名或拍一张封面照片，AI 识别书籍后生成解读，用户再填一些个人信息（身份角色、当下具体问题），生成一份**个性化的深度解读+行动清单**，最后一键生成分享图。

这个案例最有价值的地方，不是产品创意本身，是一个**独立开发者从"本地能跑"到"任何人都能打开"要经历的全部真实阶段**——这条路上的坑，比写 Agent 逻辑本身更容易被低估。

阶段一：Flask + 内网服务器，验证核心逻辑（约1-2天）

技术栈：Flask 后端 + 纯前端单页 + 一个免费的多模态大模型（用于识图+生成文本）。核心设计决策：

- **只让用户输入书名或拍封面，不要求上传全文电子版**——规避版权风险，产品定位是"转化式二次创作"（类似书评人讲书），不是盗版分发。
- **缓存网络效应**：同一本书的基础解读做本地文件缓存复用，第二个读这本书的用户只需要重新生成"个性化"那一层，基础解读直接复用缓存——这是用最简单的方式实现"记忆与检索"思想在成本控制上的应用。
- **个性化强制生效的 Prompt 坑**：早期版本里，如果用户没填身份/年龄/MBTI 等画像信息，模型会把这些字段当"可选建议"直接忽略，导致个性化程度低。修复方式是把画像信息改成强制规则块，在 Prompt 里明确列出"每种画像特征必须体现在输出的哪个具体维度"，并要求模型输出前自我检查——"如果去掉这些个人信息，输出会不会完全一样？如果一样说明没有真正个性化，需要重新生成"。

- **免费模型的真实能力边界：**身份+当下具体问题这类"强信息"，个性化效果很明显；但年龄段/MBTI/沟通风格这类"软性画像"，在生成文本的"语气冷暖"上区分度很有限——这是免费模型本身的能力天花板，不是继续优化 Prompt 能解决的，及时意识到"这是模型能力问题不是提示词问题"能避免在错误方向上浪费时间。
- **分享图生成的一个技术细节：**一键生成的分享图海报最初用固定高度画布，导致文字多的时候被截断。改成动态计算：先用一个隐藏的 canvas 元素测量所有文字实际换行后的高度，累加各区块高度，据此动态确定最终画布高度。

这个阶段的部署方式是直接在一台云服务器上跑，用内网穿透工具对外暴露端口做临时演示。

阶段二：一个认知误区——"能打开=已经公网可用"（发现问题约半天，验证约1小时）

一直以为"内网直连 IP 能打开=已经是公网服务"，实际上这个 IP 是纯内网地址，只有连公司内部网/VPN 才能访问，从来没有真正

对外公开过。这个误区一直延续到明确需要"任何人都能打开的网站"这个诉求出现才被发现——**用三个不同地区的公网检测节点分别去连这个 IP，全部超时，才彻底确认了问题。**

教训：任何"部署完成"的判断，都应该有一个独立于当前网络环境的验证步骤——不能因为"我自己能打开"就认为"公网可用"，两者是完全不同的验证维度，尤其是开发环境本身就在内网/VPN 里的时候，这个误判特别容易发生且不容易自己发现。

阶段三：迁移到公网托管（约1天，含试错时间）

依次尝试了几个方案，记录失败原因避免以后重复踩坑：

1. 某云平台的本地登录方式需要占用固定端口做本地回调，但这些端口被开发工具本身占用，无法通过重启软件解决——放弃。
2. 某轻量云服务器的工具只支持"部署到已有实例"，账号下没有现成实例，需要先去控制台手动购买——放弃（不是技术问题，是工具能力边界问题）。
3. 最终选择 Serverless 全栈托管：免费公网 HTTPS 域名，不需要额外的本机授权流程，直

接用已连接的账号部署——这是最终方案。

阶段四：Flask 到 Serverless Functions 迁移踩坑（约1天）

迁移方式是先从代码托管平台把完整源码拉到本地（比手动传输代码文件更可靠），前端文件原样复制，后端的每个路由逐个改写成独立的 Serverless 函数。

踩坑一：目录结构——第一次部署把入口文件放在了 `public/` 子目录里，构建后静态资源被当成整个 workspace 原样复制，导致入口文件没有正确映射到网站根路径。修复：把入口文件移到项目根目录。**这类"平台对目录结构有隐式约定"的坑，往往没有明确的报错提示，需要对照平台文档或者已知成功案例的目录结构逐项排查。**

踩坑二：项目名配置——首次部署报错提示"缺少项目名环境变量"。排查发现需要在项目根目录额外加一个配置文件手动写死项目名。加上后识别问题解决，但因为账号下已经存在一个同名冲突的项目，实际部署落到了那个已存在项目上，不是新建的项目——最终的线上访问地址跟最初预想的项目名不一致，需要额外去更

新所有引用这个地址的地方（比如个人网站里的跳转按钮）。

阶段五：LLM API Key 配置踩坑（约半天）

部署成功后网站能打开，但生成接口返回的全是"演示占位文案"（demo_mode），因为托管平台控制台的环境变量里没有配置真实的 API Key。

权衡决策：本想引导走"控制台手动填环境变量"的标准路径，但考虑到用户体验诉求是"一次到位不用手动配"，最终方案是把 API Key **临时硬编码进代码作为兜底默认值**（环境变量 || 硬编码默认值），这样即使控制台没配置环境变量，代码里也有真实 Key 可用，功能立即生效。

重要的安全后续处理：这个硬编码方案只能用在"不会公开的私有环境"。当后续需要把代码推送到开源代码托管平台存档时，必须把硬编码 Key 从**要公开的那份代码**里去掉（改成空字符串占位+文档说明必须在控制台配置），但**线上正在跑的服务代码不受影响，继续用硬编码版本**——两份代码故意分开维护：公开存档版本更安全，线上运行版本更省事，二者服务的目的不同，不需要强行统一。

阶段六：最耗时的 排查——"看起来 是同一个问题，其 实是三个不同根 因"（约2-3天）

这是整个项目里最曲折的一段。

现象：同一本书有时返回真实解
读内容，有时返回演示占位文
案，看起来随机出现。

第一次误诊：以为是网络不稳
定，怀疑边缘节点访问模型 API 偶
发超时。"修复"方式是加了重试机
制（最多3次，单次超时20秒）。
结果：问题依然偶发出现，说明
这不是真正的根因，只是凑巧缓
解了部分情况。

第二次误诊：以为是模型响应
慢、超时阈值设太短。调大超时
阈值到更长时间。结果：还是偶
发出现，只是频率略有下降。

第三次，真正找到根因：仔细对
比"返回演示文案"和"返回真实内
容"两种情况发生时的具体请求参
数和响应内容差异，才发现真正
的问题跟"网络"或"超时"都无关
——是一段用来判断"该走真实 API
还是走 demo 兜底"的条件判断逻
辑，在个别请求路径下判断条件
写得不够严谨，导致明明配置了
真实 Key，某些特定参数组合的请
求仍会被误判触发 demo 兜底分
支。

这次排查最有复用价值的方法论：当一个问题"看起来随机"、且第一版"修复"之后依然偶发出现时，大概率说明之前的诊断方向就是错的，"重试机制""调大超时"这类通用兜底手段只是缓解了表面症状。真正定位根因的方法是把"复现"和"不复现"两种情况的完整上下文逐字段对比，而不是继续在"网络不稳定"这个假设上叠加更多补丁。

一周时间线真实分配

阶段	大致耗时	说明
MVP核心逻辑（阶段一）	1-2天	从0到能在本地/内网跑起来
发现内网限制+验证（阶段二）	半天	意外插曲，不在原计划内
迁移公网方案选型（阶段三）	1天	含2次失败尝试
Serverless迁移+目录/配置踩坑（阶段四）	1天	
API Key配置权衡（阶段五）	半天	

阶段	大致耗时	说明
疑难Bug排查（阶段六）	2-3天	全程最耗时的部分，且前两次排查都是无效劳动



给独立开发者的真实提醒："一周做出一个 AI 小产品"，核心业务逻辑往往只占整个时间的三分之一到一半，剩下大量时间花在"部署踩坑"和"疑难问题排查"上——尤其是排查类问题存在"越接近真相前，越容易被表面现象误导走弯路"的特点，提前预留足够的缓冲时间，比精确预估"每个环节要花多久"更现实。

复盘：这个案例教会我的三件事

1. "能打开"和"公网可用"是两个完全不同的验证维度，永远用独立于开发环境的方式去验证部署是否真正生效。
2. 平台的"隐式约定"（目录结构、项目名配置）往往没有清晰报错，排查这类问题最快的方式是对照一个已知成功的最小案例逐项排查，而不是死磕报错信息本身（报错信息经常是误导性的，比如"缺少环境

变量"背后真正的坑可能是目录结构)。

3. 当同一类问题反复"看起来修复了但又复发", 要警惕自己是不是一直在同一个错误的假设方向上打补丁。停下来做"复现 vs 不复现"的逐字段对比, 往往比继续加固前一版的"修复"更快找到真根因。

产品经理清单

- ☐ 我判断"部署成功"的依据, 是"我自己能打开", 还是一个独立于我当前网络环境的验证方式?
- ☐ 遇到平台报错时, 我有没有想过"报错信息本身可能是误导性的", 需要对照已知成功案例逐项排查, 而不是死磕报错文案?
- ☐ 如果一个问题"修了但又复发"了两次以上, 我是继续在同一个假设上打补丁, 还是停下来做"复现 vs 不复现"的逐字段对比?

Part 4 到这里全部结束。三个案例分别对应"企业内部工具从 0 到多入口复用""数据类 Agent 从能查到查得准的层层加固""独立开发者一周做出小产品的真实时间分配"——它们呼应了 Part 1~3 讲过的几乎所有框架, 也是这套课程最想传达的核心信息: **方法论要在真实项目里被验证过, 才真正属于你自己。**

下一 Part，我们做毕业设计——把学到的一切，收束成一份真正拿得出手的产品 Pitch，并亲手跑通一个多智能体协作 Capstone 项目。

章节自测（5道单选，答案见文末）

Q1. 作者发现"内网能打开≠公网可用"的验证方法是什么？ A. 自己再打开一次确认 B. 用三个不同地区的公网检测节点分别连接测试 C. 问同事能不能打开 D. 查看代码逻辑

Q2. 疑难Bug排查中，前两次"误诊"为什么无效？ A. 因为完全没做任何修复 B. 网络不稳定/超时太短都只是缓解表面症状，真正原因是条件判断逻辑不严谨 C. 因为改错了文件 D. 因为没有重启服务

Q3. 文中总结的排查方法论是什么？ A. 一直增加超时时间就能解决 B. 问题"看起来随机"且修复后依然复发时，要停下来对比复现vs不复现的完整上下文 C. 换一个新项目重写 D. 忽略偶发问题

Q4. API Key硬编码作为兜底方案的安全处理原则是什么？ A. 永远不能硬编码 B. 私有环境可临时硬编码保证功能，但公开存档代码必须去掉硬编码 C. 公开代码也可以保留真实Key D. 不需要任何区分

Q5. 一周时间线里，占比最大且最容易被低估的部分是什么？ A. 核心业务逻辑开发 B. 疑难问题排查，且前几次排查方向错误导致无效劳动 C. UI设计 D. 需求调研

► [点击查看参考答案](#)